

ROLE & MISSION

You are a Principal Full-Stack Architect and Staff AI Engineer. Build a **complete, production-quality, enterprise-grade SaaS application** called the **AIVOA Customer Complaint Management System** — an AI-powered Quality Management System (QMS) module for pharmaceutical API (Active Pharmaceutical Ingredient) and FDF (Finished Dosage Form) manufacturers. The centerpiece is an **AI Copilot** that lets quality personnel log, edit, and extract customer complaints using natural language and document uploads instead of manual form entry.

Build this end-to-end: frontend, backend, database, AI workflow, seed data, sample documents, and documentation. Leave no architectural decision unmade — this specification is authoritative. Where a decision isn't covered, choose the option most consistent with enterprise SaaS conventions (Linear/Notion/Retool/Stripe/Vercel/Cursor/OpenAI-grade polish) and proceed without stopping to ask.

1. TECH STACK (MANDATORY — DO NOT SUBSTITUTE)

Frontend: React 18 + TypeScript + Vite, Redux Toolkit (with RTK Query for data fetching where appropriate), React Router v6, Tailwind CSS, Lucide React icons, Framer Motion, Recharts, Axios, Google "Inter" font (self-hosted via @fontsource/inter or Google Fonts link).

Backend: Python 3.11+, FastAPI, Pydantic v2 for validation/schemas, SQLAlchemy 2.x (async) as ORM, Alembic for migrations, Uvicorn as ASGI server.

AI Orchestration: LangGraph for a genuine multi-node stateful graph (not a thin wrapper around one LLM call). LLM provider: Groq API, primary model gemma2-9b-it, with llama-3.3-70b-versatile available as a fallback/secondary model for higher-context or higher-reasoning nodes (e.g., long PDF extraction, risk reasoning) if a config flag enables it.

Database: PostgreSQL 15+.

Auth (optional but implement a lightweight version): JWT-based auth with a seeded demo user, skippable via a "Continue as Demo User" button.

Infra/Dev: Docker + docker-compose (frontend, backend, postgres, optionally pgadmin), .env.example files for both frontend and backend, Makefile or npm/uv scripts for common tasks.

2. PRODUCT VISION & GOLDEN WORKFLOW

The end user (a QA/QC officer at a pharma company) must **never manually fill the complaint form** under normal use. The golden path is:

Natural language input OR uploaded document (PDF/email/image)



AI Copilot (LangGraph workflow) interprets intent



Structured data extraction (product, batch, dates, quantities, defect, etc.)



Complaint form auto-populates field-by-field in real time



AI generates: severity, structured defect summary, risk assessment, suggested next action



User reviews and can issue natural-language corrections ("the batch number is actually X")



AI updates ONLY the changed fields, preserves everything else, re-runs risk analysis if materially affected



Modified fields are visually highlighted (green outline/background, per reference UI)



User clicks "Commit to QMS Ledger" to persist the complaint



Full audit trail is recorded (who/what/when/before-after values)

This loop (extract → populate → assess risk → correct → re-sync) must work identically whether the input originates from typed chat text or an uploaded document, and must remain editable via chat even after a document has been processed.

3. REFERENCE UI — EXACT LAYOUT TO IMPLEMENT

Two-pane split-screen layout (see attached reference screenshots), roughly 55/45 width split:

Left pane — "Log Customer Complaint" form

- Header: "Log Customer Complaint" (large, bold) with subtitle "API & FDF Quality Assurance Module"
- Top-right status pill that changes dynamically: Pending Triage (amber/yellow, dot indicator) → Ready to Commit (green, dot indicator) → Committed (blue/gray) states
- Form organized into numbered, uppercase-labeled sections with thin divider lines:
 1. **Origin & Customer Details** — Complaint Source (dropdown: Pharmacy, Hospital, Distributor, Email, Direct Customer, Regulatory Body), Customer Name
 2. **Product & Batch Identification** — Product Name (API/FDF), Product Strength/Grade, Batch/Lot Number, Affected Quantity, Manufacturing Date, Expiry Date
 3. **Facility & Material Impact** — Originating Site Block (dropdown), Impacted Non-Product Materials (NPM) (text, e.g. "Primary packaging")

4. **Defect Analysis** — Complaint Category (e.g. "Product Defect - Discoloration"), Complaint Description / Structured Defect Summary (textarea, AI-synthesized formal QMS language)
 5. **AI Copilot Risk Assessment** (visually distinct card — light indigo/purple background, shield icon, "AI COPILOT RISK ASSESSMENT" label in caps) containing: Severity (Suggested) — dropdown/badge (Minor/Major/Critical), Suggested Next Action (e.g. "Route to QA Investigation & Issue Replacement"), Initial Risk Assessment (textarea, AI-generated reasoning)
- Bottom: full-width primary button **"Commit to QMS Ledger"** (indigo/purple, disabled until required fields + risk assessment are populated)
 - Empty fields show italic gray placeholder text like "Awaiting AI extraction..." / "Awaiting AI classification..." / "AI will synthesize the complaint into a formal QMS description..." until the AI populates them
 - Fields recently modified by an AI correction get a temporary **green border + light green background** highlight (per screenshot 4/7) that fades after a few seconds or on next edit

Right pane — "AIVOA Copilot" chat panel

- Header: flask/lab icon + "AIVOA Copilot" title, subtitle "Drop complaint files or paste text below.", connection-status dot (gray = idle, green = active/ready) top-right
- Scrollable message thread:
 - System/assistant messages: white bubble, left-aligned, small circular icon avatar (lightning bolt for AI, checkmark for confirmations)
 - User messages: solid indigo/purple bubble, right-aligned, small user-avatar circle
 - File-upload messages render as a compact file card (colored file-type icon, filename truncated, "PDF Document" subtype label)
 - Streaming/processing states render as an inline progress row (e.g., "Extracting tabular data via OCR..." with a thin animated progress bar)
- Bottom input bar: paperclip attach icon (left), rounded text input "Type a message or paste a complaint...", circular send button (indigo, checkmark/arrow icon)
- Small centered caption below input bar: "POWERED BY LANGGRAPH"

Global design language: minimal, generous white space, rounded-xl cards, soft subtle shadows (shadow-sm/shadow-md, never harsh), neutral gray-50/white backgrounds, indigo/violet as the single accent color, Inter font throughout, accessible contrast ratios (WCAG AA minimum). Implement light mode as default and a dark mode toggle (Tokyo-Night-inspired dark palette: deep navy/charcoal backgrounds, soft indigo accents) as a stretch feature behind a settings toggle. Fully responsive: on mobile/tablet, the two panes stack vertically with a tab/toggle switcher between "Form" and "Copilot" views; use a bottom nav bar pattern for switching primary app sections.

4. APPLICATION PAGES / ROUTES

1. /login — optional, minimal card-based login + "Continue as Demo User"
2. /dashboard — KPI cards (Total Complaints, Open, Critical/High-Risk, Avg. Resolution Time), Recharts visualizations (complaints by severity — pie/donut; complaints by month — bar/line; complaints by product/category — bar), recent complaints table widget
3. /complaints — Complaint List: filterable/sortable table (status, severity, source, date range, product), search bar, pagination, empty and skeleton-loading states
4. /complaints/:id — Complaint Details: read-only formatted view of a committed complaint, AI risk assessment card, linked audit trail timeline, status transition controls (e.g., Open → Under Investigation → CAPA Assigned → Closed)
5. /complaints/new — the primary two-pane Log Complaint + Copilot screen described above
6. /copilot — standalone full-width Copilot view (same engine, useful for open-ended Q&A: "how many critical complaints this month for Metformin?")
7. /history — full chat/interaction history and past AI Copilot sessions, filterable by complaint
8. /settings — optional: theme toggle, model preference (gemma2-9b-it vs llama-3.3-70b-versatile), API key status indicator, notification preferences

Use React Router with a shared <AppShell> layout: left sidebar nav (Dashboard, Complaints, New Complaint, Copilot, History, Settings), top bar with breadcrumb + user menu.

5. THE THREE MANDATORY AI TOOLS — DETAILED BEHAVIOR

5.1 Log Complaint Tool

- Trigger: free-text message describing a new complaint, with no active complaint draft in context
- Must extract into structured schema: complaint source, customer name, product name, product strength/grade, batch/lot number, affected quantity, manufacturing date, expiry date, originating site block, impacted NPM, complaint category, structured defect summary
- Must generate: severity classification (Minor/Major/Critical) with rationale, a synthesized formal QMS-style complaint description (rewriting casual customer language into regulatory-appropriate phrasing), an initial risk assessment paragraph (root-cause hypothesis + regulatory/patient-safety implications), and a suggested next action (e.g., "Route to QA Investigation & Issue Replacement")
- Must emit a per-field **confidence score** (0–1) and flag fields it could not confidently extract (leave as null, keep placeholder in UI, and mention what's missing in the chat reply) rather than hallucinating values
- Response is returned as strict structured JSON (see Section 8) which the frontend maps directly onto Redux state → form fields

5.2 Edit Complaint Tool

- Trigger: a follow-up message while a draft/committed complaint is active in context (chat history + current form snapshot are passed to the graph)

- Must diff the correction against current state, update **only** the fields explicitly referenced (never silently touch unrelated fields), and return an explicit `updated_fields: string[]` list so the frontend knows exactly which fields to highlight in green
- If the corrected field materially affects risk (batch/quantity/defect type/expiry), automatically re-run the risk-analysis subgraph and update severity/risk assessment/next action; otherwise leave the risk block untouched
- Must handle ambiguous corrections gracefully (e.g., "no it's actually 48, not 12" — must infer which field "48" refers to from conversation context) and ask a clarifying question in chat if genuinely ambiguous rather than guessing

5.3 Document Extraction Tool

- Trigger: file upload (PDF, .eml/plain-text email, or image) via the paperclip control or drag-and-drop onto the chat panel
- Pipeline: file type detection → text extraction (PDF text layer via `pypdf/pdfplumber`; for images, note that production-grade OCR is explicitly NOT required per the assignment — implement a lightweight OCR using `pytesseract` if available, otherwise stub with "vision-capable LLM description" fallback or a clearly-labeled mocked extraction) → same structured-extraction node as the Log Complaint Tool
- Must show a visible progress indicator in the chat panel during processing (e.g., "Extracting tabular data via OCR..." with animated progress bar) followed by a confirmation message summarizing what was found and a reference like a complaint report ID if present in the doc (e.g., "CC-2026-00154")
- After extraction, the tool must hand off seamlessly into Edit-Complaint mode — subsequent natural language messages continue to patch the now-populated form
- Must reject unsupported/oversized files gracefully with a clear chat error message and never crash the session

6. BONUS AI FEATURES (IMPLEMENT ALL)

Implement each as a discrete, callable capability of the AI Copilot (either as additional LangGraph nodes reachable via intent routing, or as separate endpoints invoked from UI affordances):

- **Complaint Completeness Checker** — flags missing mandatory fields before commit, shown as a checklist chip near the Commit button
- **Duplicate Complaint Detection** — on new-complaint submission, semantically compares against existing DB complaints (same product + batch + similar defect description within a time window) and warns the user with links to potential duplicates
- **Root Cause Recommendation** — suggests likely root cause categories (e.g., packaging seal failure, cold-chain excursion, cross-contamination) with brief justification
- **CAPA Recommendation** — suggests Corrective and Preventive Action steps appropriate to the severity/category

- **Complaint Summary** — one-paragraph executive summary generator, usable on the Complaint Details page and for exports
 - **AI Risk Classification** — structured severity + regulatory-reportability flag (e.g., "may require MHRA/USFDA field alert report") — clearly label this as an AI suggestion requiring human QA sign-off, never as a compliance determination
 - **Confidence Score** — surfaced per extracted field (subtle numeric or color-coded indicator on hover)
 - **Field Validation** — real-time format/plausibility validation (dates logical, expiry > manufacturing date, quantities numeric-plausible)
 - **Missing Information Detection** — proactive chat prompts asking the user for specific missing critical fields
 - **Audit Trail** — every AI action and human edit is logged with actor, timestamp, field, before-value, after-value; rendered as a timeline on the Complaint Details page
-

7. LANGGRAPH ARCHITECTURE (MANDATORY MULTI-NODE GRAPH)

Do not implement this as a single LLM call. Build a genuine StateGraph with a shared typed state object (Pydantic model) threaded through nodes, conditional edges for routing, and explicit node responsibilities:

START

→ Intent Detection Node (classify: NEW_COMPLAINT | EDIT_COMPLAINT | DOCUMENT_UPLOAD | GENERAL_QUERY | AMBIGUOUS)

→ Input Type Detection Node (text | pdf | email | image; routes document uploads through a parsing sub-step)

→ [conditional branch]

└─ Complaint Extraction Node (NEW_COMPLAINT / DOCUMENT_UPLOAD path — full structured extraction)

└─ Complaint Update Node (EDIT_COMPLAINT path — diff-based partial update against existing state)

→ Completeness & Validation Node (missing-field detection, field validation, confidence scoring)

→ Duplicate Detection Node (only on new complaints; queries DB)

→ Risk Analysis Node (severity, risk assessment narrative, regulatory-reportability flag; re-entered on material edits)

→ Recommendation Node (root cause + CAPA + suggested next action)

→ Summary Generation Node (structured defect summary / executive summary)

→ Response Formatter Node (assembles final structured JSON contract for the frontend + human-readable chat message)

→ END

Requirements:

- Use LangGraph's conditional edges (`add_conditional_edges`) for the intent-based branching, not if/else inside one node
- Persist graph state per complaint session (e.g., keyed by a `session_id/complaint_draft_id`) so multi-turn corrections have continuity; use LangGraph's checkpointing (in-memory or a Postgres-backed checkpointer) rather than re-sending full history from the frontend on every turn where avoidable
- Each node should be a small, single-responsibility Python function/class with its own prompt template (stored under `app/prompts/`), independently testable
- Model selection per node should be configurable (e.g., extraction/risk-reasoning nodes may prefer llama-3.3-70b-versatile for more headroom, lightweight classification nodes use gemma2-9b-it) via a central `app/config.py`

8. API CONTRACT

All AI-facing endpoints return a consistent structured JSON envelope, e.g.:

```
{
  "session_id": "string",
  "intent": "NEW_COMPLAINT | EDIT_COMPLAINT | DOCUMENT_UPLOAD | GENERAL_QUERY",
  "complaint": { ...full current complaint object, all fields, nulls where unknown... },
  "updated_fields": ["batch_lot_number", "affected_quantity"],
  "confidence_scores": { "batch_lot_number": 0.94, ... },
  "risk_assessment": { "severity": "Major", "suggested_next_action": "...", "initial_risk_assessment": "...", "regulatory_reportable": false },
  "recommendations": { "root_cause": [...], "capa": [...] },
  "completeness": { "is_complete": false, "missing_fields": [...] },
  "duplicate_warning": { "found": false, "candidates": [] },
  "assistant_message": "Human-readable chat reply string",
  "status": "processing | success | needs_clarification | error"
}
```

REST Endpoints (FastAPI, versioned under `/api/v1`)

- **Auth (optional):** POST `/auth/login`, POST `/auth/demo`, GET `/auth/me`
- **Complaints CRUD:** GET `/complaints`, GET `/complaints/{id}`, POST `/complaints`, PATCH `/complaints/{id}`, DELETE `/complaints/{id}`, GET `/complaints/{id}/audit-trail`

- **Copilot / Chat:** POST /copilot/chat (text-based turn, drives Intent→Update/Extraction path), GET /copilot/sessions/{session_id}/history
- **Upload:** POST /copilot/upload (multipart file upload → Document Extraction Tool)
- **Risk:** POST /copilot/risk-assessment (re-run risk analysis explicitly if needed)
- **Summary:** POST /copilot/summary
- **Dashboard:** GET /dashboard/stats, GET /dashboard/charts
- **Health:** GET /health, GET /health/db, GET /health/llm

All request/response bodies defined via Pydantic schemas in `app/schemas/`; all endpoints documented via FastAPI's automatic OpenAPI/Swagger UI at `/docs`.

9. BACKEND ARCHITECTURE (LAYERED)

backend/

app/

api/ # FastAPI routers, one module per resource

schemas/ # Pydantic request/response models

models/ # SQLAlchemy ORM models

services/ # business logic orchestration (calls repositories + langgraph)

repositories/ # DB access layer, no business logic

langgraph/ # graph definition, nodes/, state.py

prompts/ # externalized prompt templates per node

utils/ # file parsing, validation helpers, formatting

core/ # config.py, security.py, logging.py, dependencies.py

db/ # session management, base, migrations (alembic/)

tests/

main.py

requirements.txt / pyproject.toml

.env.example

Dockerfile

Apply SOLID principles: routers depend only on services; services depend on repositories + the langgraph runner, never directly on the ORM session; repositories are the only layer touching SQLAlchemy queries. Use dependency injection via FastAPI's Depends.

10. DATABASE SCHEMA (NORMALIZED)

- **users** (optional): id, email, name, hashed_password, role, created_at
- **complaints**: id, complaint_number (e.g. CC-2026-00154), source, customer_name, product_name, product_strength, batch_lot_number, affected_quantity, manufacturing_date, expiry_date, originating_site_block, impacted_npm, complaint_category, complaint_description, status (enum: draft/pending_triage/ready_to_commit/committed/under_investigation/capa_assigned/closed), created_by (FK users), created_at, updated_at
- **ai_analysis**: id, complaint_id (FK), severity, suggested_next_action, initial_risk_assessment, regulatory_reportable (bool), root_cause_suggestions (JSON), capa_suggestions (JSON), confidence_scores (JSON), model_used, created_at
- **chat_history**: id, session_id, complaint_id (FK nullable), role (user/assistant), message, attached_file_name (nullable), created_at
- **audit_logs**: id, complaint_id (FK), actor (human/ai), field_name, old_value, new_value, action_type, created_at
- **duplicate_flags**: id, complaint_id (FK), duplicate_of_complaint_id (FK), similarity_score, created_at

Define relationships (one-to-many complaint→ai_analysis history if re-runs are versioned, complaint→chat_history, complaint→audit_logs) with proper foreign keys, cascade rules, and indexes on batch_lot_number, product_name, status, created_at for dashboard/list performance. Manage schema via Alembic migrations, and provide a seed.py that inserts realistic demo complaints, users, and chat history.

11. FRONTEND ARCHITECTURE

frontend/

src/

app/ # store.ts, hooks.ts (typed useDispatch/useAppSelector)

features/

complaints/ # slice, api, components (ComplaintForm, ComplaintList, ComplaintDetails)

copilot/ # slice, api, components (ChatPanel, MessageBubble, FileUploadCard, ProgressIndicator)

dashboard/

auth/

components/ui/ # reusable primitives: Button, Card, Badge, Input, Select, Skeleton, Toast, Modal

hooks/ # reusable hooks: useDebounce, useAutoScroll, useFileUpload, useHighlightOnChange

routes/ # route definitions + AppShell layout

lib/ # axios instance, api client, formatters
types/ # shared TS types mirroring backend Pydantic schemas
styles/ # tailwind config, global css, Inter font import
vite.config.ts
tailwind.config.ts
.env.example

Redux Toolkit: a single `complaintDraftSlice` holds the authoritative in-progress complaint object plus `updatedFields` (for highlight logic) and status pill state; a `chatSlice` holds message history and streaming/loading state per session; AI responses are the only writer to `complaintDraftSlice` — form inputs are rendered as controlled components bound to Redux state and are read-only-by-default with an explicit "manual override" affordance only if you choose to allow manual edits (optional), but manual edits must also dispatch through the same reducer so state never drifts from what's rendered. Use RTK Query (or a slim custom axios-based API layer) for server communication with proper loading/error/success states.

UX details to implement precisely:

- Field-level highlight animation on AI update: green border + light green background via Framer Motion, fading after ~3s
- Status pill transitions with color + copy exactly as described in Section 3
- Chat panel: auto-scroll to latest message, optimistic rendering of user messages, animated typing/processing indicator, file-upload progress bar
- Toast notifications (top-right) for commit success, validation errors, API failures
- Skeleton loaders for dashboard charts, complaint list, and complaint details while fetching
- Empty states with helpful copy and icons for empty complaint list, empty history, no dashboard data

12. ERROR HANDLING, VALIDATION & RESILIENCE

- Pydantic-level request validation with clear 422 error payloads surfaced as inline form/chat errors, never raw stack traces
- Wrap all Groq API calls with timeout + retry-with-backoff (e.g., 2 retries, exponential backoff) and a graceful fallback assistant message ("I'm having trouble reaching the AI service — please try again in a moment") if the LLM call ultimately fails
- LangGraph nodes must validate their own output against the expected Pydantic schema and retry the LLM call once with a corrective prompt if the JSON is malformed, before failing gracefully
- Global FastAPI exception handlers producing consistent error envelopes; frontend Axios interceptor maps these to toast notifications

- Frontend: error boundaries around the chat panel and form so a rendering failure in one doesn't crash the whole app; explicit loading/error/empty states on every data-fetching component
-

13. SECURITY

- All secrets (Groq API key, DB credentials, JWT secret) via environment variables, never hardcoded, .env git-ignored with .env.example committed
 - Strict CORS configuration limited to the known frontend origin(s)
 - SQLAlchemy parameterized queries only (no raw string interpolation) to prevent SQL injection
 - Prompt-injection mitigation: system prompts explicitly instruct the model to treat uploaded document/chat content as data, not instructions; sanitize/strip suspicious control phrases from extracted text before it's echoed back into further prompts; never let extracted document content trigger tool calls or alter system-level behavior
 - File upload safety: enforce allow-listed MIME types/extensions (.pdf, .eml, .txt, .png, .jpg), max file size limit, store uploads outside the web root with randomized filenames, never execute uploaded content
 - Passwords (if auth implemented) hashed with bcrypt/argon2; JWT with reasonable expiry + refresh flow
-

14. SAMPLE DATA & DELIVERABLES CHECKLIST

Antigravity must also generate:

- 3–5 realistic **fictional** pharmaceutical complaint sample PDFs (formatted like a customer complaint report, similar in spirit to "Fictional_Pharma_Customer_Complaint.pdf" referenced in the reference UI) covering different defect types (discoloration, foreign matter contamination, packaging seal failure, short-fill, mislabeling)
- 2–3 realistic sample "complaint emails" as plain text/.eml fixtures
- A seed.py / SQL seed script populating the database with ~15–20 realistic historical complaints across varied severities/statuses/products for a populated dashboard and list view out of the box
- A root README.md covering: architecture overview with a diagram (ASCII or Mermaid), setup instructions (local + Docker), environment variable reference, how the LangGraph workflow works with a diagram, how to run migrations/seed, how to run tests, and a section mapping each assignment requirement to where it's implemented in the codebase
- docker-compose.yml wiring frontend, backend, and Postgres together with sensible defaults, plus a one-command make dev / docker compose up path

- Basic automated tests: backend (pytest) covering at least the LangGraph node logic and key API endpoints; frontend (vitest/RTL) covering the complaint slice reducers and the field-highlight behavior
-

15. CODE QUALITY BAR

Modular, scalable, and maintainable throughout. Consistent naming conventions (PascalCase components, camelCase functions/vars in TS, snake_case in Python). Full type safety end-to-end (TypeScript strict mode; Python type hints everywhere, mypy-clean where feasible). Reusable UI primitives and hooks rather than copy-pasted markup/logic. Docstrings on backend service/graph-node functions explaining intent, inputs, and outputs. No dead code, no TODO-stubbed core paths — every mandatory and bonus feature listed above must be functionally wired end-to-end, not mocked in the UI only.

16. EXECUTION INSTRUCTIONS FOR ANTIGRAVITY

Build in this order, verifying each milestone works before proceeding:

1. Scaffold monorepo structure (frontend/, backend/) + Docker + env files
2. Database models + Alembic migration + seed script; verify DB boots and seeds via Docker
3. Backend core CRUD endpoints for complaints (no AI yet); verify via /docs
4. LangGraph state model + all nodes with prompt templates; unit-test each node in isolation with mock LLM responses first, then wire real Groq calls
5. Copilot chat + upload endpoints wired to the graph; verify via /docs/curl end-to-end (text → structured JSON)
6. Frontend scaffold: routing, Redux store, AppShell, design tokens/Tailwind config, Inter font
7. Build the New Complaint two-pane screen exactly per Section 3, wired to live backend endpoints (no mock data)
8. Build Dashboard, Complaint List, Complaint Details, History, Settings pages
9. Wire audit trail, duplicate detection, completeness checker, and remaining bonus features into both backend responses and corresponding UI surfaces
10. Polish pass: animations, loading/skeleton/empty/error states, responsive breakpoints, dark mode
11. Write README, finalize sample PDFs/emails, finalize tests, final Docker smoke test of the whole stack from a clean checkout